

Encapsulation

In Java, access modifiers control the visibility and accessibility of your classes, methods, variables, and constructors. They are the foundation of **encapsulation**, which allows you to secure your data and hide the inner workings of your code.

Here is a quick overview of how the three main modifiers, plus the "default" behavior, restrict access:

Access Modifier	Inside Same Class	Inside Same Package	Subclasses (Different Package)	The World (Everywhere)
<code>public</code>	✓ Yes	✓ Yes	✓ Yes	✓ Yes
<code>protected</code>	✓ Yes	✓ Yes	✓ Yes (via inheritance)	✗ No
<i>default (no keyword)</i>	✓ Yes	✓ Yes	✗ No	✗ No
<code>private</code>	✓ Yes	✗ No	✗ No	✗ No

Public (`public`)

The public modifier provides the **least restrictive** level of access. Anything declared as public can be seen and used by any other class in your entire project. [1, 2]

- **When to use:** Use this for your main API endpoints, major application services, helper utilities, or anything designed to be shared globally across packages.

Private (`private`)

The private modifier provides the **highest level of restriction**. Variables or methods marked private are hidden from every other class in your project—even classes inside the exact same folder (package). They can only be accessed within the specific class where they are defined. [1, 2, 3]

- **When to use:** Use this by default for almost all of your class variables (fields). You then expose them safely using public "getter" and "setter" methods if other classes need to read or change them.

Protected (protected)

The protected modifier is tailored specifically for inheritance. It allows a parent class to share its components with its children (subclasses), even if those child classes are located in completely different folders or packages. Additionally, any *other* class inside the same package can also see it. [1, 2]

- When to use: Use this when building a framework or class hierarchy where child classes genuinely need direct access to a parent's methods or data to extend its functionality.

Note on "Default" (Package-Private)

If you do not type public, private, or protected next to a variable or method, Java automatically assigns it **default** access. This means the code is visible to everything inside the **same package**, but entirely hidden from the outside world. [1, 2, 3]

Would you like to see a practical example of **how these modifiers interact** across different packages, or do you want to explore how **getters and setters** protect private fields? [1]

Here is a complete, working example

Using two different packages (animals and zoo) to show exactly how all four access levels behave across your project

Package 1: animals

First, let's create a base class named Animal that defines a variable for each of the four access levels. And look another class inside that **same package** to see what it can access.

```
package animals;

public class Animal {
    // 1. Public: Accessible from anywhere in the project
    public String publicName = "Global Lion";

    // 2. Protected: Accessible in the same package AND by subclasses
    protected String protectedSound = "Roar";
    // 3. Default (No modifier): Accessible ONLY in the same package

    String defaultColor = "Golden";
    // 4. Private: Accessible ONLY inside this Animal class

    private String privateId = "ID-9942";

    public void testInsideClass() {
        // All of these work perfectly inside the same class
        System.out.println(publicName); // ✓ Success
        System.out.println(protectedSound); // ✓ Success
        System.out.println(defaultColor); // ✓ Success
        System.out.println(privateId); // ✓ Success
    }

    // ● The Getter Method: Anyone can call this to READ the ID
    public String getPrivateId() {
        return this.privateId;
    }
}
```

```

package animals;

public class PackageCompanion {

    public void testSamePackage() {
        Animal animal = new Animal();
        System.out.println(animal.publicName); // ✓ Success
        System.out.println(animal.protectedSound); // ✓ Success
        System.out.println(animal.defaultColor); // ✓ Success

        // System.out.println(animal.privateId); // ✗ COMPILE ERROR: privateId is private
        // to Animal
    }
}

```

Package 2: zoo

Now, let's move outside the animals package to see how inheritance and external access restrictions work.

```

package zoo;
import animals.Animal; // Must import since it's a different package

public class Lion extends Animal {
    public void testSubclass() {
        // 1. Public works everywhere
        System.out.println(publicName); // ✓ Success

        // 2. Protected works because Lion EXTENDS Animal (Inheritance)
        System.out.println(protectedSound); // ✓ Success

        // 3. Default fails because Lion is in a DIFFERENT package
        // System.out.println(defaultColor); // ✗ COMPILE ERROR: Not visible outside package
        // 4. Private fails because it belongs exclusively to Animal
        // System.out.println(privateId); // ✗ COMPILE ERROR: Not visible outside Animal
    }
}

```

Run the code and see how the modifiers react.

This Main class is in a completely separate package named mainrunner to simulate "the rest of the world" looking into your code.

```
package mainrunner;

import animals.Animal; // Import the base class
import zoo.Lion;      // Import the subclass

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Testing from an unrelated Main Class ---");
        Animal genericAnimal = new Animal();

        // 1. Public works everywhere
        System.out.println("Public Field: " + genericAnimal.publicName); //  Success

        // 2. Protected fails here (Main is not a subclass of Animal)
        // System.out.println(genericAnimal.protectedSound); //  COMPILE ERROR

        // 3. Default fails here (Main is in a different package)
        // System.out.println(genericAnimal.defaultColor); //  COMPILE ERROR

        // 4. Private fails completely
        // System.out.println(genericAnimal.privateId); //  COMPILE ERROR

        System.out.println("\n--- Testing via the Lion Subclass ---");
        Lion simba = new Lion();

        // This triggers the method inside Lion, which can safely
        // read the public and protected fields due to inheritance
        simba.testSubclass();
    }
}
```

What Happens When You Run This?

If you uncomment any of the red-marked lines (✖), your code will refuse to compile. Left as it is above, it compiles flawlessly and prints out:

```
text
--- Testing from an unrelated Main Class ---
Public Field: Global Lion

--- Testing via the Lion Subclass ---
Global Lion
Roar
```

Getter and Setter method

A **getter method** is a public method used to safely retrieve (get) the value of a hidden, private variable from outside its class.

By making class variables private, you prevent outside classes from accidentally changing or corrupting your data. The getter method acts as a controlled, **read-only window** to the world.

```
package mainrunner;
import animals.Animal;

public class Main {
    public static void main(String[] args) {
        Animal genericAnimal = new Animal();

        // ❌ This still causes a compile error:
        // System.out.println(genericAnimal.privateId);

        // ✅ This works perfectly!
        String idValue = genericAnimal.getPrivateId();
        System.out.println("Accessed via Getter: " + idValue);
    }
}
```

Why do this instead of just making the variable public?

1. **Read-Only Protection:** By providing only a getter (and no setter), you make the variable completely immutable to the outside world. Other classes can see it, but they can **never change it**.
2. **Data Logging & Control:** You can add logic inside the getter. For example, you could log a message every time a sensitive piece of data is accessed:

A **setter method** is a public method used to safely update or change (set) the value of a hidden, private variable from outside its class.

While getters provide read-only access, setters provide **write-only access**. By routing all changes through a setter method, your class can inspect, validate, or reject new values before updating the actual variable.

Why use a Setter instead of a public variable?

- **Data Validation:** You can block bad inputs (like preventing a `setAge(int age)` method from accepting negative numbers).
- **Encapsulation:** The internal storage logic is hidden. If you later decide to change how IDs are structured (e.g., adding a prefix automatically), you only change the code *inside* the setter, without breaking anyone else's code.

```
package animals;
public class Animal {
    .....
    // 🟢 The Setter Method: Anyone can call this to CHANGE the ID
    public void setPrivateId(String newId) {
        // Validation: Ensure the new ID isn't empty or null
        if (newId != null && !newId.trim().isEmpty()) {
            this.privateId = newId; // 'this.privateId' refers to the private field above
        } else {
            System.out.println("Error: Invalid ID provided. Update rejected.");
        }
    }
}
```

```
package mainrunner;
import animals.Animal;
public class Main {
    public static void main(String[] args) {
        Animal genericAnimal = new Animal();
        // ❌ This causes a compile error:
        // genericAnimal.privateId = "NEW-ID-123";

        // ✅ This works perfectly!
        genericAnimal.setPrivateId("NEW-ID-123");

        // ❌ This will trigger our validation rule and keep the old ID
        genericAnimal.setPrivateId("");
    }
}
```